LABPATTERNS



Ingeniería del Software II

Hecho por: Oiher Bernet, Julen Haro y Oihane Zubiaurre

1. Simple Factory

Vulnerabilidades encontradas:

A. ¿Qué sucede si aparece un nuevo síntoma (por ejemplo, mareos)?

Tendríamos que modificar los métodos createSymptom de las clases Covid19Pacient y Medicament. Esto viola el principio OCP ya que estaríamos modificando el código al añadir un nuevo síntoma.

B. ¿Cómo se puede crear un nuevo síntoma sin cambiar las clases existentes (principio OCP)?

Creando una nueva clase SymptomFactory con el método createSymptom() de esa manera las clases Covid19Pacient y Medicament delegaran la creación llamando a SymptomFactory.createSymptom(name).

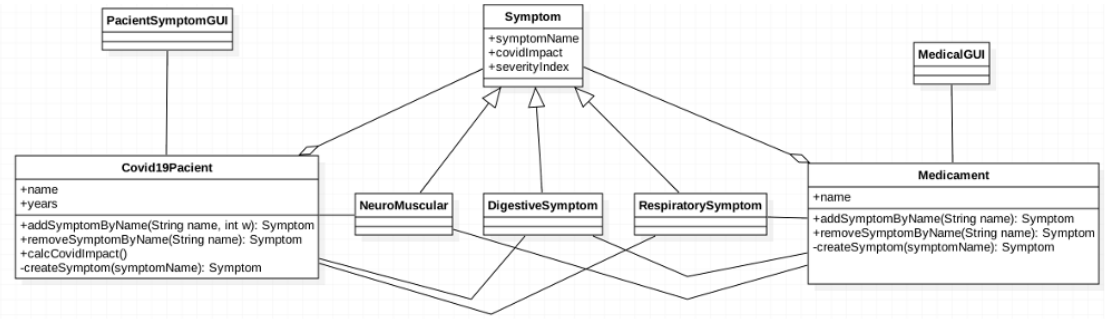
C. ¿Cuántas responsabilidades tienen las clases de Covid19Pacient y Medicament (principio SRP)?

- Covid19Pacient: Vemos 4 responsabilidades (Gestión de datos personales del paciente, Gestión de la colección de síntomas, Notificación de cambios y lógica de negocio).
- Medicament: Vemos dos responsabilidades (Gestión de datos y colección de síntomas y Delegación de creación).

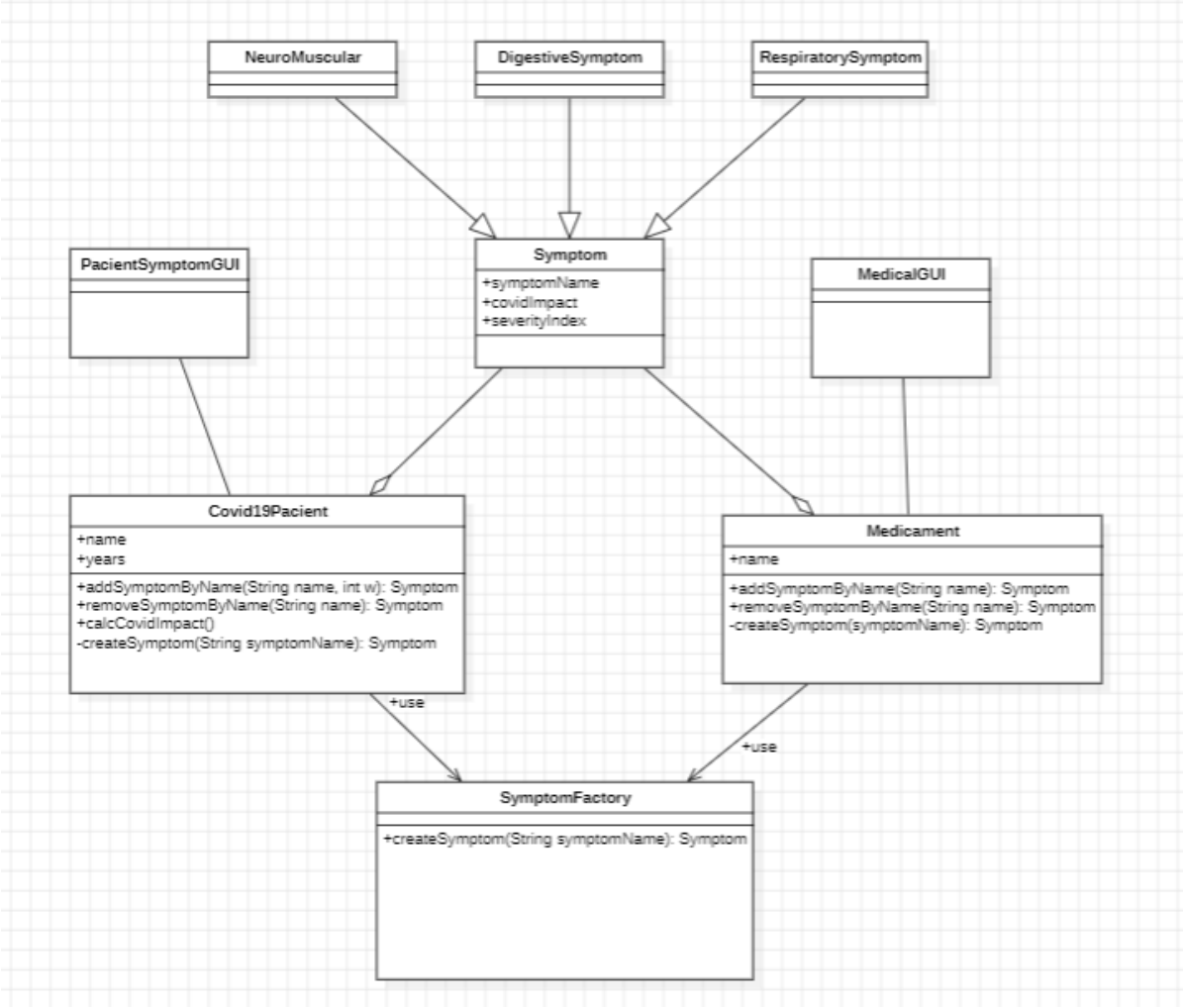
Tareas para realizar:

1. Realiza un nuevo diseño de la aplicación (diagrama UML) aplicando el patrón Simple Factory para eliminar vulnerabilidades anteriores y mejorar el diseño en general. Describe con claridad los cambios realizados.

Antes:



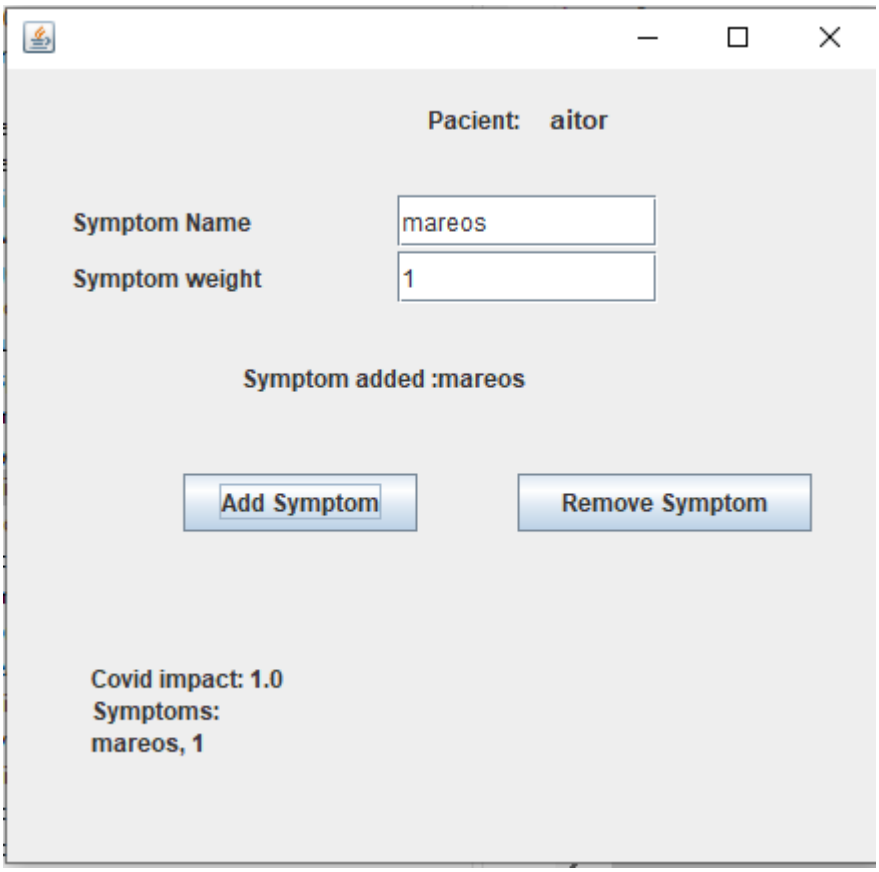
Despues:



Se crea una clase SymptomFactory que tiene el método createSymptom. De esa manera todas las clases que deseen crear un síntoma delegaran su creación a esa clase.
Antes al añadir un nuevo síntoma había que modificar el código de Covid19Pacient y Medicament mientras que ahora esas clases quedan cerradas a modificación y el sistema queda abierto a extensión. Si se quiere

añadir un nuevo síntoma únicamente se tendrá que modificar SymptomFactory (OCP).
Además, se libera a las clases de dominio de la responsabilidad de conocer cómo se instancian los síntomas (SRP).

2. Implementa la aplicación y agrega el nuevo síntoma "mareos" asociado a un tipo de impacto 1.



3. Cómo se puede adaptar la clase Factory, para que los objetos Symptom que utilicen las clases Covid19Pacient y Medicament sean únicos. Es decir, para cada síntoma sólo exista un objeto. (Si hay x síntomas en el sistema, haya únicamente x objetos Symptom)

Para garantizar que los objetos Symptom sean únicos en todo el sistema, se debe aplicar una caché o registro dentro de la clase SymptomFactory.

Esto se implementa añadiendo un atributo de tipo Map<String, Symptom> que almacene las instancias ya creadas.

Con este codigo lo hacemos

```
package factory;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import domain.DigestiveSymptom;
import domain.NeuroMuscularSymptom;
import domain.RespiratorySymptom;
import domain.Symptom;

public class SymptomFactory {



private Map<String, Symptom> symptomCache = new HashMap<>();

public Symptom createSymptom(String symptomName) {



if (symptomCache.containsKey(symptomName)) {
return symptomCache.get(symptomName);
}

 List<String> impact5 = Arrays.asList("fiebre", "tos seca", "astenia","expectoracion");
 List<Double> index5 = Arrays.asList(87.9, 67.7, 38.1, 33.4);
 List<String> impact3 = Arrays.asList("disnea", "dolor de garganta", "cefalea","mialgia","escalofrios");
 List<Double> index3 = Arrays.asList(18.6, 13.9, 13.6, 14.8, 11.4);
 List<String> impact1 = Arrays.asList("nauseas", "vómitos", "congestión
nasal","diarrea","hemoptisis","congestion conjuntival", "mareos");
 List<Double> index1 = Arrays.asList(5.0, 4.8, 3.7, 0.9, 0.8, 0.5, 2.0);

 List<String> digestiveSymptom=Arrays.asList("nauseas", "vómitos","diarrea");
 List<String> neuroMuscularSymptom=Arrays.asList("fiebre", "astenia", "cefalea",
"mialgia","escalofrios","mareos");
 List<String> respiratorySymptom=Arrays.asList("tos seca","expectoracion","disnea","dolor de
garganta", "congestión nasal","hemoptisis","congestion conjuntival");

 int impact=0;
 double index=0;
 if (impact5.contains(symptomName)) {impact=5; index=
index5.get(impact5.indexOf(symptomName));}
 else if (impact3.contains(symptomName)) {impact=3;index=
index3.get(impact3.indexOf(symptomName));}
 else if (impact1.contains(symptomName)) {impact=1; index=
index1.get(impact1.indexOf(symptomName));}

 Symptom s = null;
 if (impact!=0) {
        if (digestiveSymptom.contains(symptomName))
```

```
                s = new DigestiveSymptom(symptomName, (int)index, impact);
        else if (neuroMuscularSymptom.contains(symptomName))
                s = new NeuroMuscularSymptom(symptomName, (int)index, impact);
        else if (respiratorySymptom.contains(symptomName))
                s = new RespiratorySymptom(symptomName, (int)index, impact);
}


if (s != null) {
        symptomCache.put(symptomName, s);
}

return s;
}
}
```

2. Patrón Observer

Tareas para realizar:

1. Cambia el programa principal para crear 2 pacientes Covid19Pacient con sus interfaces PacientSymptomGUI y PacientObserverGUI.

Esta es el código que tenemos:

```
package observer;

import java.util.Observable;

import adapter2.ShowPacientTableGUI; import domain.Covid19Pacient; import
factory.SymptomFactory;

public class Main {

/** * Launch the application. */

public static void main(String args[]){

 Covid19Pacient pacient=new Covid19Pacient("aitor", 35,new SymptomFactory());

    new PacientObserverGUI (pacient);

 new PacientSymptomGUI (pacient);

 new PacientThermometerGUI(pacient);

 ShowPacientTableGUI gui=new ShowPacientTableGUI (pacient);

    gui.setPreferredSize(new java.awt.Dimension(300,200));

gui.setVisible(true);

 Covid19Pacient pacient2=new Covid19Pacient("ane", 22,new SymptomFactory());
new PacientObserverGUI (pacient2);

new PacientSymptomGUI (pacient2);

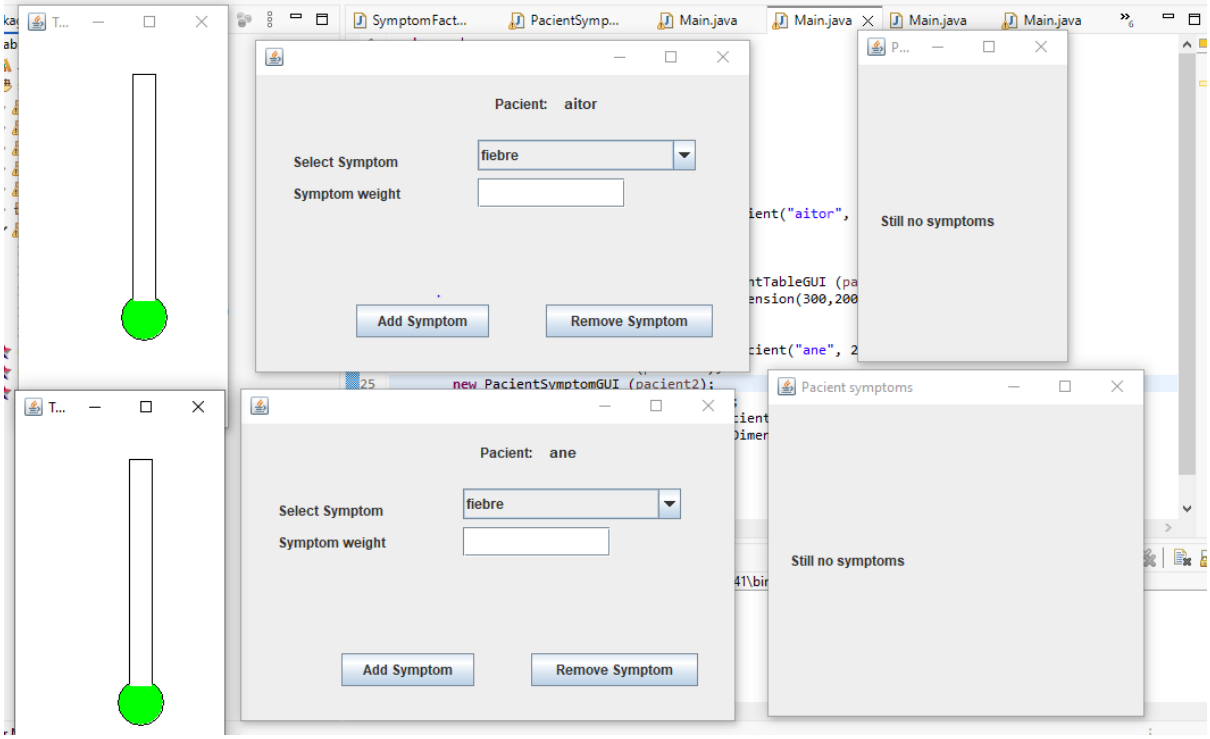
new PacientThermometerGUI(pacient2);

ShowPacientTableGUI gui2=new ShowPacientTableGUI (pacient2);
gui2.setPreferredSize(new java.awt.Dimension(300,200));

    gui2.setVisible(true);

}
```

}


SymptomFact...
PacientSymp...
Main.java
Main.java
Main.java
Main.java
Pacient: aitor
Select Symptom
fiebre
Symptom weight
Add Symptom
Remove Symptom
Still no symptoms
Pacient: ane
Select Symptom
fiebre
Symptom weight
Add Symptom
Remove Symptom
Pacient symptoms
Still no symptoms

3. Patrón Adapter

Tareas para realizar:

1. Añade el código necesario en la clase Covid19PacientTableModelAdapter y ejecuta la aplicación para comprobar que funciona correctamente.

Este es el codigo de Covid19PacientTableModelAdapter:

```
package adapter2;

import java.util.Vector;

import javax.swing.table.AbstractTableModel;

import domain.Covid19Pacient;
import domain.Symptom;

public class Covid19PacientTableModelAdapter extends AbstractTableModel {
  protected Covid19Pacient pacient;
  protected String[] columnNames =
    new String[] {"Symptom", "Weight" };
  protected Vector<Symptom> sintomas;

  public Covid19PacientTableModelAdapter(Covid19Pacient p) {
    this.pacient=p;
    sintomas=new Vector<Symptom>(p.getSymptoms());
  }

  public int getColumnCount() {
```

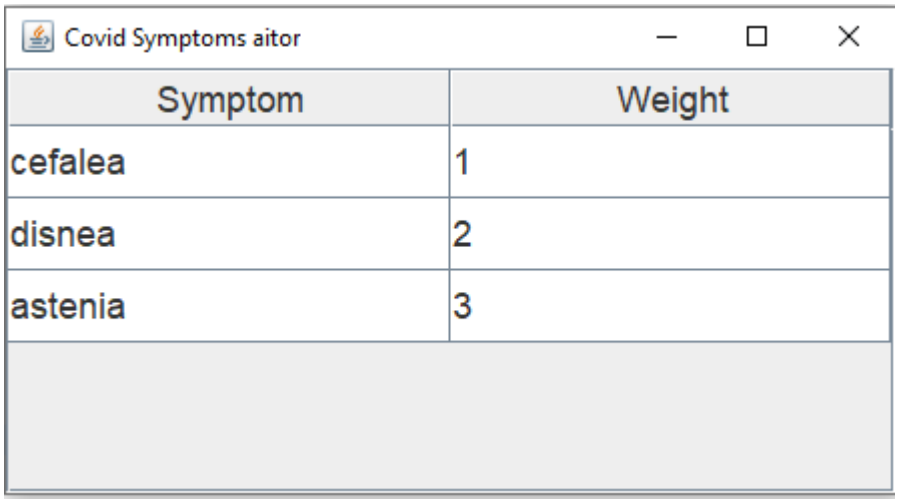
```
    // Challenge!
  return 2;
  }

  public String getColumnName(int i) {
    // Challenge!
  return columnNames[i];
  }

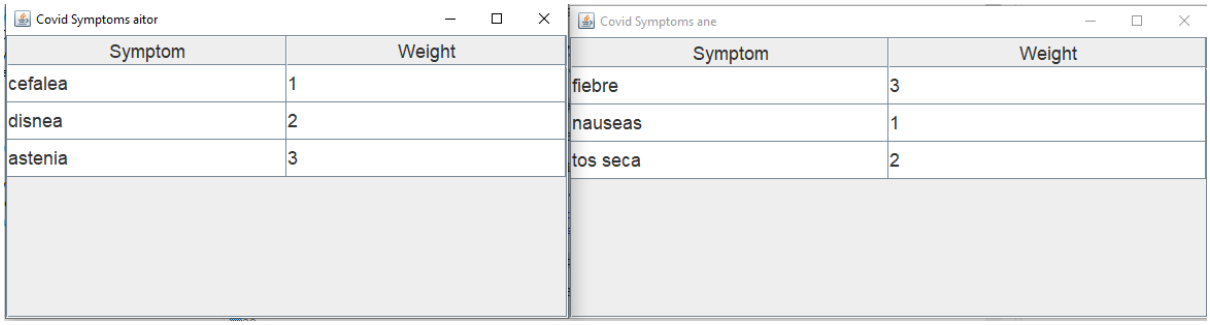
  public int getRowCount() {
    // Challenge!
  return pacient.getSymptoms().size();
  }

  public Object getValueAt(int row, int col) {
  sintomas=new Vector<Symptom>(pacient.getSymptoms());
    // Challenge!
  if(col==0)
  return sintomas.get(row).getName();

  if(col==1)
  return pacient.getWeight(sintomas.get(row));
  return null;
  }
}
```

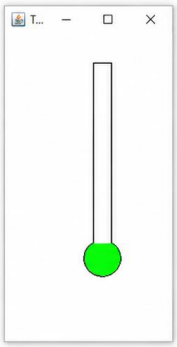


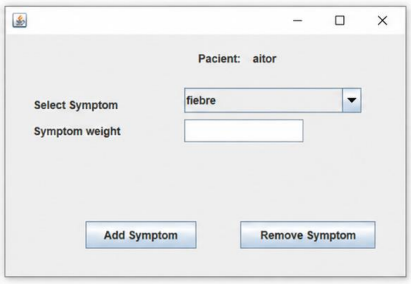
2. Añade otro paciente con otros síntomas, y ejecuta la aplicación para que aparezcan los 2 pacientes con sus síntomas.

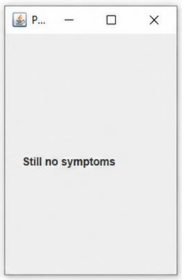


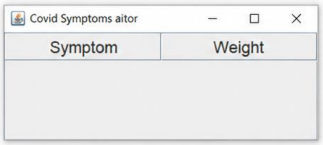
3. Cómo podrías añadir esta nueva pantalla al ejercicio anterior del observer, de forma que cada vez que se añada un nuevo síntoma a un paciente, se actualice la tabla.

Esto lo hemos mediante implementando observer en Covid19PacientTableModelAdapter

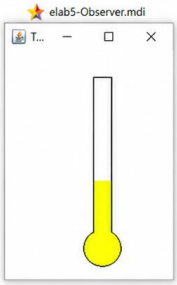


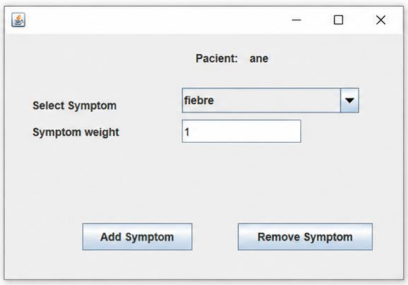


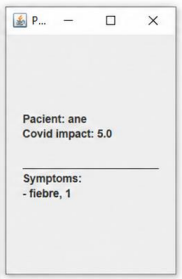


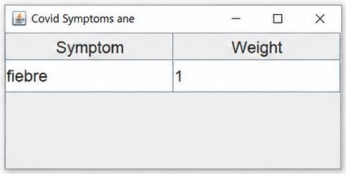


elab5-Observer.mdi









4. Patrón Iterator y Adapter

Tareas para realizar:

1. Crea un paciente Covid19Pacient con cinco síntomas. La clase Covid19Pacient NO PUEDE CAMBIARSE NADA.

```java
package adapter;

import java.util.Comparator;
import java.util.Iterator;

import domain.Covid19Pacient;
import factory.SymptomFactory;

public class Main {

    public static void main(String[] args) {
        Covid19Pacient pacient = new Covid19Pacient("aitor", 35, new SymptomFactory());

        pacient.addSymptomByName("disnea", 2);
        pacient.addSymptomByName("cefalea", 1);
        pacient.addSymptomByName("astenia", 3);
        pacient.addSymptomByName("mialgia", 5);
        pacient.addSymptomByName("tos seca", 4);

        InvertedIterator ii = new Covid19PacientIterAdapter(pacient);

        Comparator<Object> compName = new NameComparator();
        Comparator<Object> compSeverity = new SeverityComparator();

        Iterator<Object> i = Sorting.sortedIterator(ii, compName);
        while(i.hasNext()) {
            System.out.println(i.next());
        }

        ii.goLast();

        i = Sorting.sortedIterator(ii, compSeverity);
        while(i.hasNext()) {
            System.out.println(i.next());
        }
    }
}
```

2. Implementa las interfaces Comparator: una para la ordenación por symptomName, y otra para la ordenación según severityIndex.

- NameComparator:

```
package adapter;

import java.util.Comparator;
import domain.Symptom;

public class NameComparator implements Comparator<Object> {

@Override
public int compare(Object arg0, Object arg1) {
String s1 = ((Symptom) arg0).getName();
String s2 = ((Symptom) arg1).getName();

return s1.compareTo(s2);
}
}
```

- SeverityComparator:

```
package adapter;

import java.util.Comparator;
import domain.Symptom;

public class SeverityComparator implements Comparator<Object> {

@Override
public int compare(Object arg0, Object arg1) {
int s1=((Symptom)arg0).getSeverityIndex();
int s2=((Symptom)arg1).getSeverityIndex();

if(s1>s2)return 1;
if(s1<s2)return -1;
return 0;

}

}
```

3. Crea el patrón adapter sobre la clase Covid19Pacient, implementando la interfaz InvertedIterator. Recuerda crear una constructora adecuada para enviarle la información del paciente.

- Covid19PacientIterAdapter:

```
package adapter;
```

```java
import java.util.List;
import java.util.Vector;
import domain.Covid19Pacient;
import domain.Symptom;

public class Covid19PacientIterAdapter implements InvertedIterator {
private List<Symptom> symptoms;
private int position;

public Covid19PacientIterAdapter(Covid19Pacient pacient) {
this.symptoms = new Vector<Symptom>(pacient.getSymptoms());
goLast();
}

@Override
public void goLast() {
position = symptoms.size() - 1;
}

@Override
public boolean hasPrevious() {
return position >= 0;
}

@Override
public Object previous() {
if (hasPrevious()) {
return symptoms.get(position--);
}
return null;
}
}
```

Vista del resultado:

```
astenia
cefalea
disnea
mialgia
tos seca
disnea
mialgia
cefalea
tos seca
astenia
```

Como vemos el resultado está bien ya que nos lo ordena por nombre y tambien por severidad/gravedad.